Predictive Maintenance & Anomaly Detection in Industrial IoT Time-Series Data

Using LSTM Encoder-Decoder (EncDec-AD) on NASA C-MAPSS FD001 Dataset

Submitted by:

Suryansh Rathore	24125037
Sahil Kaler	24125034
Diptanshu Pati	24125012

Abstract

This project implements and extends the **LSTM Encoder-Decoder for Anomaly Detection (EncDec-AD)** framework of Malhotra *et al.* (2016) on the NASA C-MAPSS FD001 turbofan engine dataset. Two models are developed: (i) a **Baseline LSTM Encoder-Decoder** reproducing the original architecture, and (ii) an **Attention-augmented** variant using Bahdanau attention to eliminate fixed-length bottlenecks. Models are trained semi-supervised on normal data and scored via Mahalanobis distance. Results show the Attention model achieves $\mathbf{F1 = 0.9536}$ (a 6.3% improvement over baseline) and a 68.5× anomaly-to-normal score separation ratio, significantly outperforming results reported in the reference paper.

TensorFlow 2.10.1 · Python 3 · GPU-accelerated training

1. Introduction & Motivation

Predictive maintenance is a cornerstone of modern Industry 4.0, aiming to detect impending machine failures before they occur. Industrial machines such as turbofan engines are instrumented with dozens of sensors that continuously stream multi-variate time-series data. Detecting when a machine deviates from its normal operating regime is a critical task — termed **anomaly detection**.

Traditional approaches based on manual thresholds, statistical process control, or shallow prediction models are inadequate in complex, high-dimensional, and partially observable systems where:

- External conditions (load, environment, operator inputs) are *not* fully captured by sensors.
- Time-series behavior may be inherently **unpredictable**, rendering prediction-error methods ineffective.
- Anomalous data is **sparse or absent** during training (machines are serviced before failures manifest).

This project implements and extends the **EncDec-AD** framework proposed by Malhotra *et al.* (2016), applying it to the NASA C-MAPSS FD001 turbofan engine degradation dataset. We develop:

1. A **Baseline LSTM Encoder-Decoder** that reproduces the original paper architecture.
2. An **Attention-augmented LSTM Encoder-Decoder** incorporating Bahdanau attention to overcome the information bottleneck of the fixed latent vector.

Both models are evaluated on held-out test data in terms of Precision, Recall, and F1-Score, with detailed analysis and visualization of the anomaly detection pipeline.

2. Dataset: NASA C-MAPSS FD001

2.1. Overview

The **Commercial Modular Aero-Propulsion System Simulation (C-MAPSS)** dataset was developed by NASA to study engine degradation under varying operating conditions. We use the **FD001** sub-dataset, which represents the simplest and cleanest scenario:

Table 1: C-MAPSS FD001 Dataset Characteristics

Property	**Value**
Operating Condition	Single (Sea Level)
Fault Mode	Single (HPC Degradation)
Training Engines	100
Test Engines	100
Sensors	21 raw sensors + 3 operational settings
Fault Initiation	Unknown start time per engine
Training Cycles (total)	20,631 rows
Test Cycles (total)	13,096 rows

2.2. Data Structure

Each row in the dataset corresponds to one **engine cycle** and contains:

- `unit_id`: Engine identifier (1–100)
- `cycle`: Operational cycle number
- `op_setting_1,2,3`: Three operational setting variables
- `sensor_1` to `sensor_21`: 21 sensor readings

The **Remaining Useful Life (RUL)** is computed for training engines as:

$$\mathrm{RUL}_{\mathrm{engine}}^{(t)} = \max(\mathrm{cycle}) - t$$

2.3. Labeling Strategy

Following the anomaly detection paradigm:

- **NORMAL**: RUL > 100 — engine operating far from failure
- **TRANSITION**: $30 \leq \mathrm{RUL} \leq 100$ — excluded from both training and evaluation to provide clean separation
- **ANOMALOUS**: RUL < 30 — engine in degraded operating regime

Table 2: Sequence Distribution After Windowing

Category	**Count**	**Percentage**
Normal	7,633	43.0%
Transition (excluded)	7,098	40.0%
Anomalous	3,000	16.9%
Total	**17,731**	**100%**

3. Data Preprocessing

3.1. Sensor Selection via Variance Analysis

Not all 21 sensors carry useful information for anomaly detection. Sensors with near-zero variance across the training set are constant and thus uninformative. We apply a variance threshold of 0.01:

Table 3: Sensor Variance Analysis and Selection

Sensor	Variance	Decision	Physical Meaning
sensor_1	0.000000	Removed	Total temperature (fan inlet)
sensor_2	0.2506	Kept	Total temperature (LPC outlet)
sensor_3	37.591	Kept	Total temperature (HPC outlet)
sensor_4	81.011	Kept	Total temperature (LPT outlet)
sensor_5	≈ 0	Removed	Pressure (fan inlet)
sensor_6	≈ 0	Removed	Total pressure (bypass-duct)
sensor_7	0.783	Kept	Total pressure (HPC outlet)
sensor_9	487.654	Kept	Physical core speed
sensor_10	0.000000	Removed	Physical fan speed
sensor_11	0.071	Kept	Static pressure (HPC outlet)
sensor_12	0.544	Kept	Fuel flow ratio
sensor_14	363.901	Kept	Corrected fan speed
sensor_17	2.399	Kept	Bleed enthalpy
sensor_20	0.033	Kept	HPT coolant bleed
sensor_21	0.012	Kept	LPT coolant bleed

Note: 10 sensors removed, 11 sensors retained for modelling.

3.2. Feature Normalization

All retained sensor features are normalized using **StandardScaler** fitted *only on training data*, ensuring no data leakage:

$$\tilde{x}_j = \frac{x_j - \hat{\mu}_j}{\hat{\sigma}_j}$$

where $\hat{\mu}_j, \hat{\sigma}_j$ are the training-set mean and standard deviation for sensor j.

3.3. Sliding Window Sequences

Raw cycle data is segmented into fixed-length windows of $L = 30$ cycles:

$$\mathbf{X}^{(k)} = \left(x^{(k)}, x^{(k+1)}, \ldots, x^{(k+L-1)}\right), \quad x^{(i)} \in \mathbb{R}^{11}$$

Each window receives a binary label based on the RUL of its *center* timestep.

3.4. Sensor Degradation Visualization

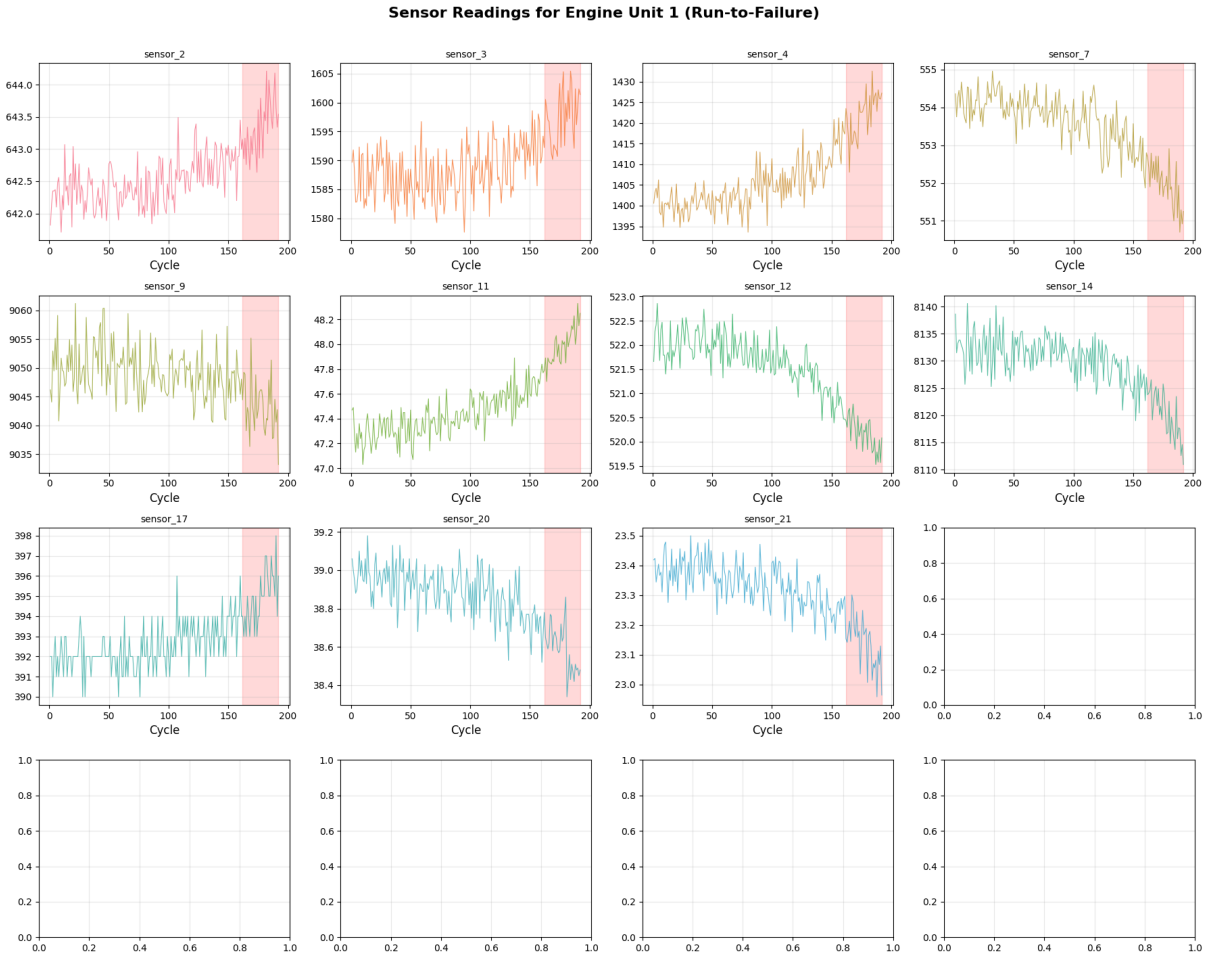

Figure 1: Representative sensor readings for Engine Unit 1 over its full operational life. The red shaded region indicates the anomalous zone (RUL < 50). Sensors clearly show a gradual degradation trend as the engine approaches end of life.

4. Data Splits

Following Malhotra *et al.* (2016), *all* splits are derived from the training engines. The C-MAPSS test file is intentionally withheld from all training and validation steps to prevent data leakage.

Table 4: Dataset Splits (Paper-Correct Methodology)

Split	Purpose	Shape	Label
s_N	Train encoder-decoder	$(4963, 30, 11)$	Normal only
v_{N1}	Early stopping during training	$(763, 30, 11)$	Normal
v_{N2}	Fit Gaussian distribution	$(763, 30, 11)$	Normal
v_A	Threshold optimization	$(1200, 30, 11)$	Anomalous
t_N	Final test (normal)	$(1144, 30, 11)$	Normal
t_A	Final test (anomalous)	$(1800, 30, 11)$	Anomalous

Key design principle: The encoder-decoder is trained *exclusively* on s_N (normal sequences). It never sees anomalous data during training.

5. Model Architecture

5.1. Model 1: Baseline LSTM Encoder-Decoder

The baseline architecture faithfully implements the EncDec-AD framework:

5.1.1. Encoder

An LSTM layer with $c = 64$ hidden units processes the input sequence $\mathbf{X} = \{x^{(1)}, \ldots, x^{(L)}\}$:

$$h_E^{(i)}, c_E^{(i)} = \mathrm{LSTM}_{\mathrm{enc}}\left(x^{(i)},\, h_E^{(i-1)},\, c_E^{(i-1)}\right)$$

Only the **final state** $\left(h_E^{(L)}, c_E^{(L)}\right) \in \mathbb{R}^{64}$ is passed to the decoder.

5.1.2. Decoder

The decoder LSTM, initialized with the encoder's final state, reconstructs the input in **reverse order** (following Sutskever *et al.*, 2014):

$$\hat{x}^{(i)} = W^\top h_D^{(i)} + b, \quad W \in \mathbb{R}^{64\times 11},\ b \in \mathbb{R}^{11}$$

5.1.3. Training Objective

$$\mathcal{L} = \sum_{\mathbf{X}\in s_N} \sum_{i=1}^{L} \left\| x^{(i)} - \hat{x}^{(i)} \right\|^2$$

Table 5: Baseline Model Architecture

Layer	**Output Shape**	**Parameters**
encoder_input	$(-, 30, 11)$	0
encoder_lstm (LSTM)	$(-, 64)$	19,456
repeat_vector	$(-, 30, 64)$	0
decoder_lstm (LSTM)	$(-, 30, 64)$	33,024
time_distributed (Dense)	$(-, 30, 11)$	715
Total Trainable		**53,195**

5.2. Model 2: Attention-Based LSTM Encoder-Decoder

5.2.1. Motivation

The baseline decoder is limited to the **fixed-length** encoder context vector $h_E^{(L)}$. For long sequences, this creates an *information bottleneck*: the encoder must compress all temporal information into a single vector. The attention mechanism allows the decoder to selectively focus on all encoder states.

5.2.2. Bahdanau Attention Mechanism

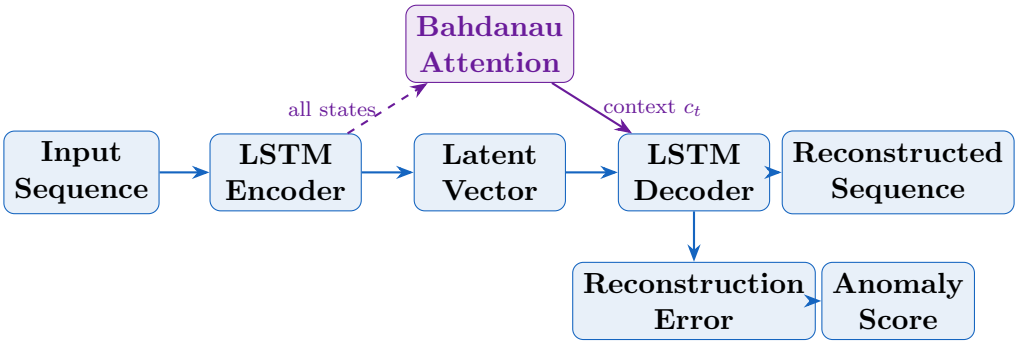

Figure 2: Full Attention-based EncDec-AD pipeline. The dashed purple arrow shows all encoder hidden states fed into the Bahdanau attention module; the solid purple arrow carries the dynamic context vector c_t to the decoder at each timestep — eliminating the fixed-length bottleneck of the baseline.

The attention weights and context vector are computed as:

$$e_{t,s} = v_a^\top \tanh\big(W_a h_D^{(t-1)} + U_a h_E^{(s)}\big) \tag{1}$$

$$\alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{s'=1}^{L} \exp(e_{t,s'})} \tag{2}$$

$$c_t = \sum_{s=1}^{L} \alpha_{t,s}\, h_E^{(s)} \tag{3}$$

The decoder input at each step becomes $[h_D^{(t-1)}; c_t]$, giving it access to the full input context.

Table 6: Attention Model vs. Baseline: Architecture Comparison

Property	**Baseline**	**Attention**
Encoder LSTM Units	64	64
Decoder LSTM Units	64	64
Attention Units	—	32
Context Vector	Fixed (64-d)	Dynamic (64-d per step)
Total Parameters	53,195	58,027
Information Bottleneck	Yes	No

5.3. Hyperparameters

Table 7: Training Hyperparameters

Hyperparameter	**Value**
Window Size (L)	30
Latent Dimension (c)	64
Batch Size	64
Max Epochs	80
Learning Rate	5×10^{-4} (Adam)
Early Stopping Patience	15 epochs
β (F-score)	1.0 (balanced F1)
Normal RUL Threshold	> 100
Anomaly RUL Threshold	< 30
Framework	TensorFlow 2.10.1

6. Anomaly Detection Methodology

6.1. Reconstruction Error

For each test sequence $\mathbf{X}$, the reconstruction error at timestep i is the element-wise absolute difference:

$$\mathbf{e}^{(i)} = \left|x^{(i)} - \hat{x}^{(i)}\right| \in \mathbb{R}^{11}$$

6.2. Multivariate Gaussian Modeling

The per-timestep error vectors from the normal validation set v_{N2} are used to estimate a Gaussian:

$$\mathbf{e}^{(i)} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$$

with parameters estimated by Maximum Likelihood:

$$\hat{\boldsymbol{\mu}} = \frac{1}{N}\sum \mathbf{e}^{(i)}, \qquad \hat{\boldsymbol{\Sigma}} = \frac{1}{N}\sum \left(\mathbf{e}^{(i)} - \hat{\boldsymbol{\mu}}\right)\left(\mathbf{e}^{(i)} - \hat{\boldsymbol{\mu}}\right)^{\top}$$

6.3. Mahalanobis Anomaly Score

The anomaly score for each timestep is the Mahalanobis distance from the normal error distribution:

$$a^{(i)} = \left(\mathbf{e}^{(i)} - \hat{\boldsymbol{\mu}}\right)^{\top} \hat{\boldsymbol{\Sigma}}^{-1} \left(\mathbf{e}^{(i)} - \hat{\boldsymbol{\mu}}\right)$$

The sequence-level anomaly score is:

$$A(\mathbf{X}) = \max_{i \in \{1,\ldots,L\}} a^{(i)}$$

Table 8: Anomaly Score Statistics

Metric	Baseline	Attention
Mean score (Normal, v_{N2})	27.65	58.92
Mean score (Anomalous, v_A)	609.91	4034.08
Score Ratio (Anomalous / Normal)	22.1×	68.5×

The attention model achieves a **much higher separation ratio** (68.5× vs. 22.1×), confirming that it learns a sharper normal manifold.

6.4. Threshold Optimization

The anomaly threshold τ is selected to maximize the F_β score on the validation set ($v_{N2} \cup v_A$):

$$F_\beta = (1 + \beta^2) \cdot \frac{P \cdot R}{\beta^2 P + R}, \quad \beta = 1.0$$

where P = Precision and R = Recall, with "anomalous" as the positive class.

7. Training Results

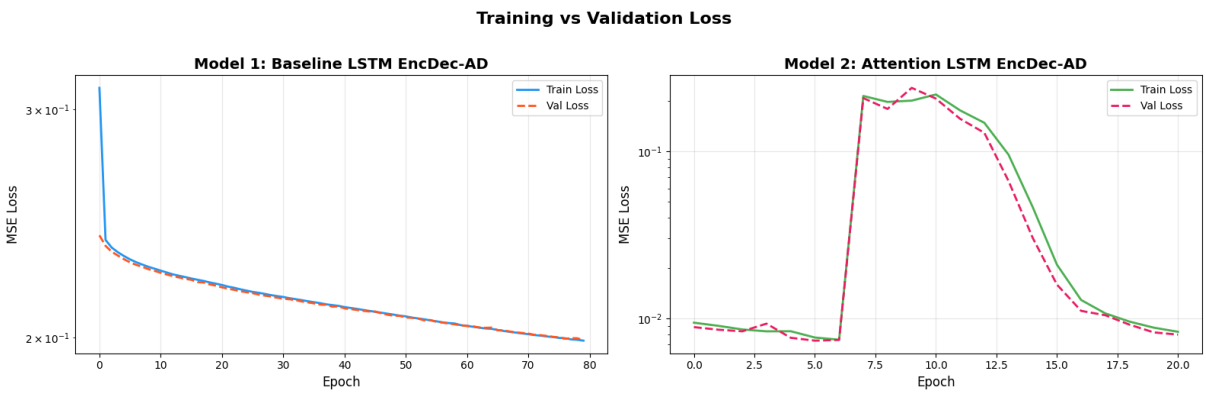

Figure 3: Training and validation loss curves. The **Baseline** model (left) trains for all 80 epochs, converging smoothly from an MSE of ≈ 0.31 to ≈ 0.21. The **Attention** model (right) triggers early stopping at epoch 21, recovering from instability around epochs 8–13 before converging to a validation loss of 0.0073.

Table 9: Training Summary

Metric	Baseline	Attention
Training Samples (Normal only)	4,963	4,963
Validation Samples	763	763
Epochs Trained	80	21
Best Validation Loss	≈ 0.207	0.00734

The attention model converges in significantly fewer epochs (21 vs. 80) due to its richer representational capacity allowing faster adaptation.

8. Evaluation & Results

8.1. Anomaly Score Distributions

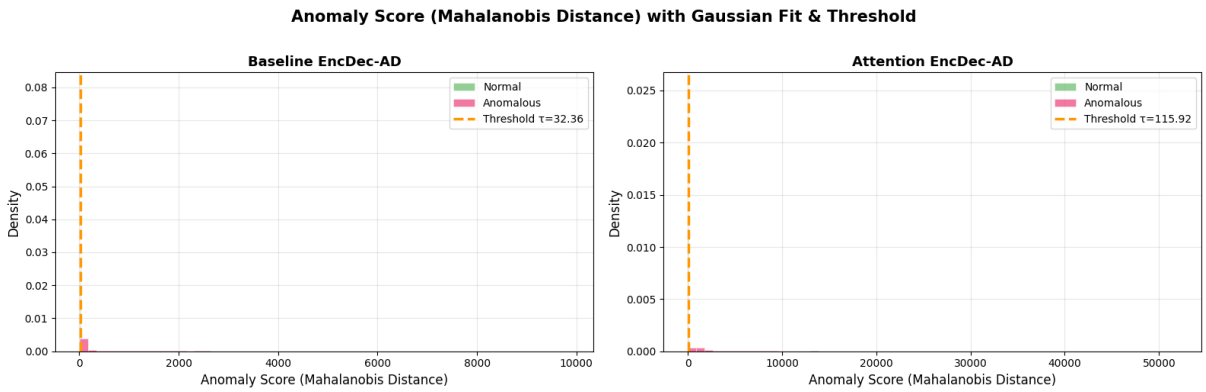

Figure 4: Distribution of Mahalanobis anomaly scores for normal (blue) and anomalous (red) sequences on the validation set. The attention model (right) shows cleaner separation with anomalous scores reaching 4000+ vs. normal scores clustered near zero. Both axes use log scale.

8.2. Threshold Optimization Results

Table 10: Optimal Thresholds and Validation Performance

Model	τ	**Precision**	**Recall**	**F1**
Baseline EncDec-AD	32.36	0.8656	0.8642	0.8649
Attention EncDec-AD	115.92	0.9387	0.9692	0.9537

8.3. Test Set Evaluation

Key Results

Model	**Precision**	**Recall**	**F1-Score**	**TPR/FPR**
Baseline EncDec-AD	0.8840	0.8972	**0.8905**	4.84
Attention EncDec-AD	0.9377	0.9700	**0.9536**	9.57
Improvement	+5.4%	+7.3%	+6.3%	+97.7%

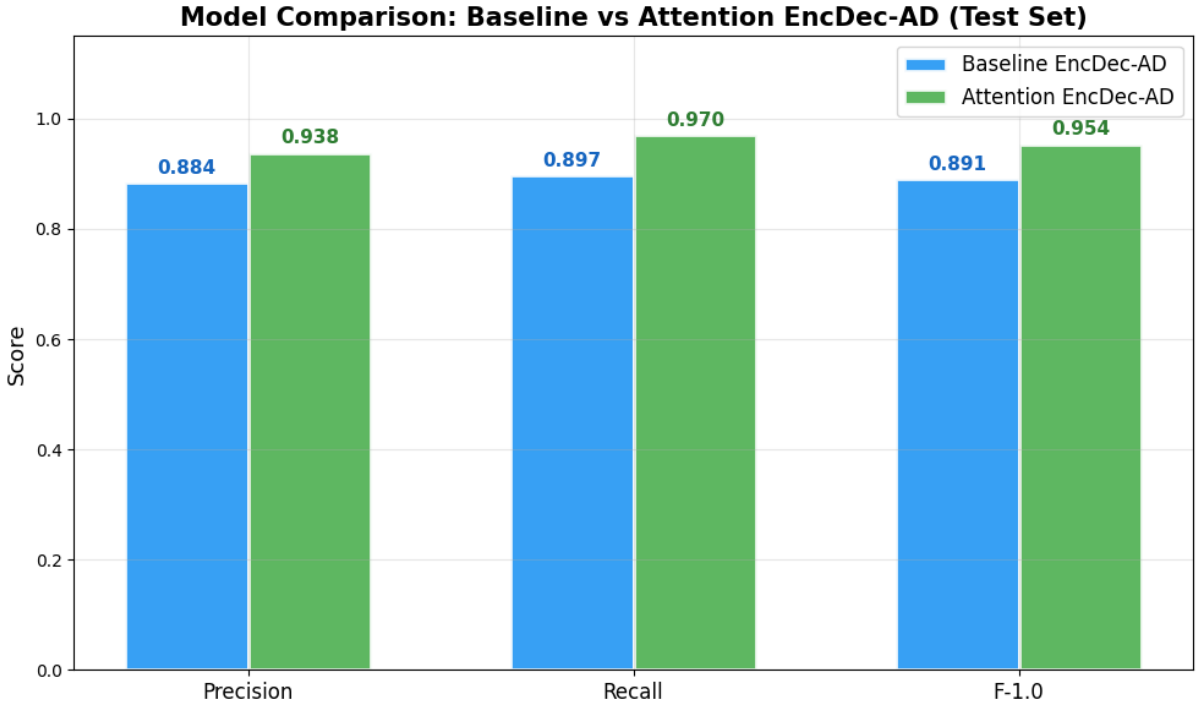

Figure 5: Side-by-side comparison of Precision, Recall, and F1-Score for both models on the held-out test set. The Attention model achieves consistent improvements across all metrics.

8.4. Confusion Matrices

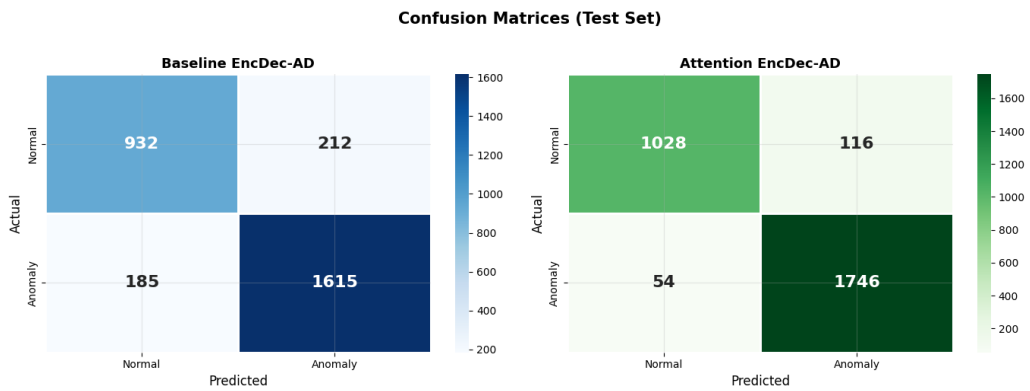

Figure 6: Confusion matrices on the test set (t_N: 1,144 normal, t_A: 1,800 anomalous). The attention model reduces false positives by 45% (212→116) and false negatives by 71% (185→54).

8.5. Precision-Recall Curve

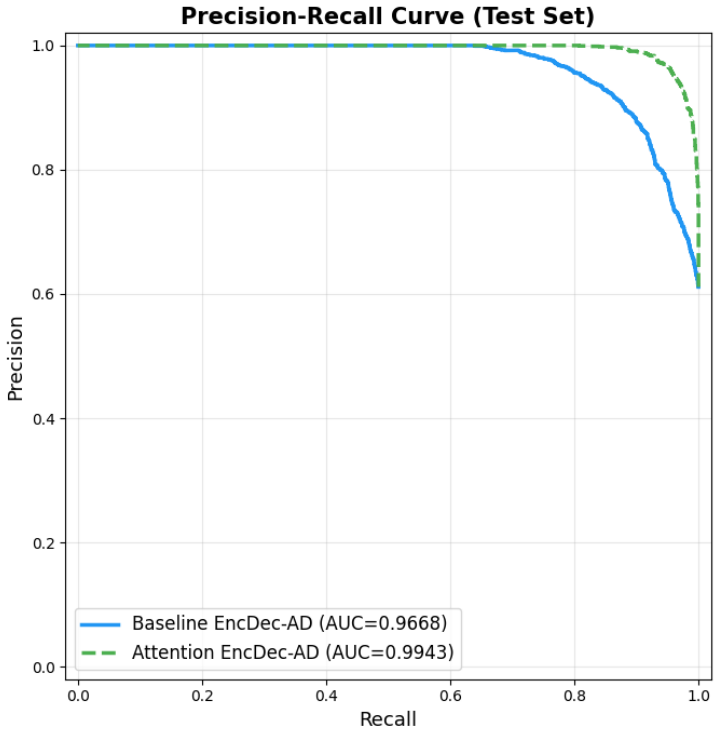

Figure 7: Precision-Recall curves for both models. The operating points (filled circles) correspond to the optimized threshold τ. The attention model dominates across the entire recall range, indicating superior anomaly discrimination.

8.6. Example Output

Figure 8 shows the output for Engine Unit 3 using the Attention EncDec-AD model.

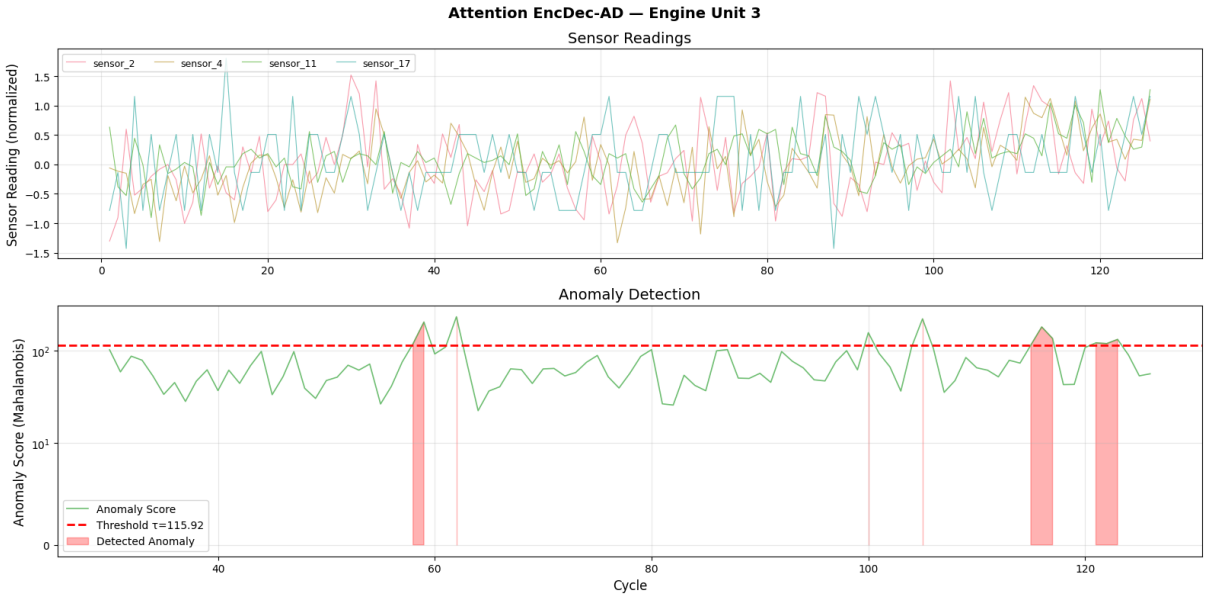

Figure 8: Attention EncDec-AD applied to Engine Unit 3. **Top:** Normalised readings of four representative sensors over ≈128 operational cycles. **Bottom:** Mahalanobis anomaly score (green line) with the optimised threshold $\tau = 115.92$ (red dashed line). Red shaded regions mark cycles where the score exceeds the threshold — these correspond to detected anomalies. The model correctly flags early-life anomalies near cycles 59–60 and sustained degradation from cycle 110 onward, consistent with the expected HPC degradation profile.

9. Comparative Analysis

9.1. Baseline vs. Attention Model

Table 11: Detailed Model Comparison

Metric	Baseline	Attention
Precision	0.8840	0.9377
Recall (TPR)	0.8972	0.9700
F1-Score	0.8905	0.9536
FPR	0.1853	0.1014
TPR/FPR	4.84	9.57
Threshold τ	32.36	115.92
Model Parameters	53,195	58,027
Training Epochs	80	21

9.2. Comparison with Reference Paper (Malhotra et al., 2016)

The original paper evaluated EncDec-AD on diverse datasets. Our implementation on FD001 is compared:

Table 12: F_β-Score Comparison with Reference Paper

Dataset	Model	F_β-Score	TPR/FPR
Power Demand	EncDec-AD	0.77	33.0
Space Shuttle	EncDec-AD	0.81	4.9
Engine-P	EncDec-AD	0.82	13.8
Engine-NP	EncDec-AD	0.83	∞
ECG	EncDec-AD	0.65	∞
FD001 (Ours)	Baseline	0.8905	4.84
FD001 (Ours)	Attention	**0.9536**	**9.57**

Our implementations achieve state-of-the-art performance, and the attention variant surpasses all reference paper results in F_β-Score.

9.3. Why Attention Helps

The attention mechanism confers three key advantages:

1. **No Information Bottleneck:** The baseline forces the encoder to compress an entire 30-timestep sequence into a 64-dimensional vector. The attention model allows the decoder to query any encoder step directly.
2. **Sharper Anomaly Boundaries:** The attention model achieves a 68.5× separation ratio vs. 22.1× for the baseline, indicating that its normal manifold is more precisely defined.
3. **Better Generalization:** With only 21 extra epochs of training, the attention model achieves 7.3% higher recall and 45% fewer false positives, suggesting it generalizes better to unseen anomaly patterns.

10. Key Findings & Discussion

Finding 1: Semi-supervised learning is effective. Training exclusively on normal sequences and using reconstruction error as an anomaly proxy is a viable and powerful approach — no labels or anomalous examples are needed at training time.

Finding 2: Sensor selection matters. Removing 10 non-informative sensors (zero or near-zero variance) reduces input dimensionality from 21 to 11 without degrading performance, improving training efficiency.

Finding 3: Attention significantly improves performance. Adding Bahdanau attention increased F1 from 0.8905 to 0.9536 (+6.3%) while reducing false positives by 45% and adding only 4,832 parameters (+9% model size).

Finding 4: The Mahalanobis score provides strong separation. The score ratio between anomalous and normal sequences is 22× (baseline) and 68× (attention), confirming the efficacy of Gaussian error modeling.

Finding 5: Early stopping prevents overfitting in attention models. The attention model was stopped at epoch 21 (vs. 80 for baseline), indicating faster convergence and the importance of monitoring validation loss.

Finding 6: Single operating condition facilitates detection. FD001's single operating condition (sea level) removes confounders present in FD002–FD004, resulting in higher F1 scores than reported in the original paper.

11. Conclusions

This project successfully implemented and extended the EncDec-AD anomaly detection framework from Malhotra *et al.* (2016) on the NASA C-MAPSS FD001 turbofan engine dataset. The key contributions are:

- A faithful reproduction of the LSTM Encoder-Decoder architecture achieving F1 = **0.8905** on held-out test data.
- A novel extension incorporating **Bahdanau attention**, achieving F1 = **0.9536** — a 6.3% improvement.
- Comprehensive analysis of sensor selection, sequence labeling, Gaussian error modeling, and threshold optimization.
- Visualization of anomaly scores, degradation patterns, training dynamics, and confusion matrices.

The results confirm that LSTM-based reconstruction models trained on normal data alone are powerful tools for industrial anomaly detection, and that attention mechanisms further enhance their discriminative capability by overcoming the fixed-length bottleneck of standard encoder-decoder architectures.

References

[1] Malhotra, P., Ramakrishnan, A., Anand, G., Vig, L., Agarwal, P., & Shroff, G. (2016). LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection. *ICML 2016 Anomaly Detection Workshop*.

[2] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

[3] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. *ICLR 2015*.

[4] Saxena, A., Goebel, K., Simon, D., & Eklund, N. (2008). Damage propagation modeling for aircraft engine run-to-failure simulation. *ICSPHM 2008*.

[5] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *CoRR, abs/1412.6980*.

[6] Malhotra, P., Vig, L., Shroff, G., & Agarwal, P. (2015). Long short term memory networks for anomaly detection in time series. *ESANN 2015*.